UNIVERSIDAD DEL VALLE DE GUATEMALA

CONSTRUCCIÓN DE COMPILADORES

Pablo



Proyecto 2. Código intermedio de Compiscript

Ihan Marroquin - 23108

Diego Patzan - 23525

GUATEMALA, 18 de octubre del 2026

Informe final

https://github.com/UVG-Tareas-y-proyectos/fase_compilacion_compis

Descripción del proyecto

En esta segunda fase extendimos el analizador de Compiscript para generar código intermedio. Partimos del árbol sintáctico y de la tabla de símbolos que construimos en la primera entrega. Cuando el código supera las validaciones, lo traducimos a instrucciones de tres direcciones o TAC. Cada instrucción representa una operación pequeña y conserva la información necesaria para una futura traducción a código de máquina.

El programa recibe archivos con extensión .cps y permite trabajar desde la terminal o desde el IDE. Se puede consultar el árbol, la tabla de símbolos, los errores, el TAC y los registros de activación. También incluimos un intérprete que ejecuta las instrucciones generadas para comprobar sus resultados. La salida principal de esta entrega es el código intermedio; todavía no generamos assembler ni un ejecutable nativo.

Arquitectura de la implementación

Mantuvimos los generadores que desarrollamos el semestre anterior. YALex utiliza Compiscript.yal para reconocer los tokens y YAPar utiliza Compiscript.yalp para construir el parser SLR. Las reducciones del parser producen el árbol que consume SemanticAnalyzer. La gramática Compiscript.g4 se conserva como referencia del lenguaje. Aunque el flujo del enunciado menciona ANTLR, nuestra implementación continúa con YALex y YAPar propios.

La fase semántica comprueba declaraciones, ámbitos, tipos, argumentos, retornos y las demás reglas del lenguaje. Además de los diagnósticos, ahora conserva qué símbolo corresponde a cada declaración, en qué entorno se encuentra cada bloque y cuál es el tipo de cada expresión. Esto evita resolver nuevamente esos datos durante la traducción.

Si no existen errores, el método generate_code de SemanticAnalyzer inicia una segunda pasada mediante TACGenerator, que hereda sus métodos auxiliares. El generador recorre el mismo árbol y aplica una regla según el nodo encontrado. Una expresión devuelve su tipo y el lugar donde quedó el resultado. El ámbito actual y los destinos de los saltos se mantienen como contexto del recorrido. Esta es la relación de la implementación con la traducción orientada por la sintaxis vista en clase.

Archivo	**Responsabilidad**
parser.py	Conectar YALex y YAPar y construir el árbol sintáctico.
semantic.py	Validar el programa y conservar atributos, símbolos y ámbitos.
symbols.py y layout.py	Administrar símbolos y asignar direcciones, tamaños y desplazamientos.
ir.py	Representar cuádruplos, funciones, programas y el pool de temporales.
tac.py	Traducir las construcciones del árbol a instrucciones TAC.
pipeline.py	Coordinar las fases y bloquear TAC cuando existen errores.

Archivo	Responsabilidad
vm.py	Interpretar los cuádruplos para comprobar resultados.
scripts/ide.py y scripts/compiscript.py	Permitir el uso gráfico y por terminal.

Diseño del código de tres direcciones

Elegimos cuádruplos porque permiten guardar cada instrucción como operador, primer argumento, segundo argumento y resultado. La clase Instruction también conserva línea y columna del código fuente, fuera de los cuatro campos. TACProgram agrupa las funciones, los descriptores de clases y el punto de entrada __main__. El archivo .tac muestra las instrucciones en texto y el archivo .json conserva su estructura y metadatos.

Una dirección puede ser un literal, una variable o un temporal. Las variables reciben nombres como v0_resultado, donde el número identifica su entorno. Los temporales usan el prefijo % y se muestran como %t1, %t2 y siguientes. Las etiquetas de salto tienen nombres como L1 y L2. El prefijo de los temporales evita confundirlos con una variable del usuario que se llame t1.

Los renglones de la siguiente tabla son ejemplos independientes de cuádruplos y no forman una secuencia de ejecución. El campo vacío indica que esa instrucción no necesita el argumento correspondiente.

Operador	Argumento 1	Argumento 2	Resultado
+	a	b	%t1
copy	%t1	vacío	v0_resultado
ifFalse	%t1	vacío	L1
store_index	%t1	2	v0_numeros
call	%t1	2	%t2

Conjunto de instrucciones

La notación mantiene las operaciones de la presentación de código intermedio. En los ejemplos siguientes, x, y y z son direcciones, L es una etiqueta, i es un índice y n es una cantidad. Las operaciones especiales de objetos, ámbitos y closures completan el modelo que necesita Compiscript.

Instrucción	Forma legible	Descripción
copy	x = y	Copiar un valor a una dirección.
+ - * / %	x = y op z	Realizar una operación aritmética.
neg	x = neg y	Cambiar el signo del operando.

Instrucción	Forma legible	Descripción
== != < <= > >=	x = y op z	Comparar valores y producir un booleano.
!	x = ! y	Negar un booleano.
i2f	x = i2f y	Convertir un entero a float.
label y goto	L: / goto L	Marcar un destino y saltar hacia él.
if e ifFalse	ifFalse x goto L	Saltar según el valor de una condición.
declare	declare x	Crear la celda de un símbolo en el ámbito actual.
enter y leave	enter E1 / leave	Entrar o salir de un ámbito léxico.
closure	x = closure F1	Guardar una función y su entorno de definición.
param	param x	Preparar un argumento para una llamada.
call	x = call f, n	Llamar una closure con n argumentos explícitos.
return	return x	Devolver un valor; puede aparecer sin operando.
print	print x	Mostrar el valor de una expresión.
class	x = class C1	Asociar un descriptor de clase con su entorno.
newobj	x = newobj clase	Crear un objeto e inicializar sus atributos.
method	x = method obj, nombre	Obtener un método ligado a su receptor.
load_field	x = obj.atributo	Leer un atributo del objeto.
store_field	obj.atributo = x	Modificar un atributo del objeto.
newlist	x = newlist n	Crear una lista de n elementos.
length	x = length lista	Obtener la cantidad de elementos.
load_index	x = lista[i]	Leer un elemento por su índice.
store_index	lista[i] = x	Escribir un elemento por su índice.
try y endtry	try L / endtry	Registrar o retirar un manejador de errores.
exception	x = exception	Recuperar el mensaje disponible en catch.

En store_field y store_index el primer argumento es el valor, el segundo es el atributo o índice y el campo resultado identifica el objeto o lista de destino. En los saltos el campo resultado contiene la etiqueta. En call, los argumentos identifican la closure y la cantidad de parámetros, mientras el resultado indica dónde guardar el retorno.

Traducción de expresiones y asignaciones

El generador sigue la precedencia y asociatividad que ya están representadas en el árbol. Primero traduce los hijos de una operación, obtiene sus lugares y tipos, solicita un temporal y emite la instrucción del padre. Por ejemplo, en la siguiente declaración se realiza primero la multiplicación:

```
let resultado: integer = a + b * c;
```

```
%t1 = b * c
%t2 = a + %t1
resultado = %t2
```

Estos ejemplos muestran la regla central. La salida real también utiliza las direcciones de los símbolos, instrucciones declare y copias de lectura. Esas copias son necesarias para conservar el orden de evaluación. Si a vale 1, la expresión a + (a = 2) produce 3, porque el operando izquierdo conserva el valor anterior de a.

Las declaraciones generan una celda y la asignación de su inicializador. Las constantes siguen la misma traducción, pero el análisis semántico impide reasignarlas. Cuando una asignación acepta integer en un destino float, se emite i2f. Para una lectura o escritura indexada se evalúan primero el receptor y el índice, y después el valor correspondiente.

Los operadores && y || utilizan cortocircuito. Si el operando izquierdo de && es falso, no se evalúa el derecho. Si el izquierdo de || es verdadero, tampoco se evalúa el derecho. El operador ternario utiliza etiquetas para ejecutar solamente la rama seleccionada. Esto conserva el comportamiento de llamadas y asignaciones que aparezcan dentro de esas expresiones.

Traducción del control de flujo

Para if/else generamos una etiqueta para la rama falsa y otra para la salida. El salto condicional decide qué bloque se ejecuta y el salto final evita entrar también al else. La regla se puede representar de esta manera:

```
ifFalse condicion goto L_else
codigo_del_if
goto L_fin
L_else:
codigo_del_else
L_fin:
```

En while la etiqueta de inicio queda antes de evaluar la condición. Al terminar el cuerpo se vuelve a esa etiqueta. En do-while la evaluación aparece después del cuerpo, por lo que este se ejecuta al menos una vez. En for separamos inicialización, condición, cuerpo y actualización. Por eso continue se dirige a la actualización y después vuelve a comprobar la condición.

```
L_condicion:
codigo_de_la_condicion
ifFalse condicion goto L_fin
codigo_del_cuerpo
goto L_condicion
L_fin:
```

Mantenemos pilas con los destinos de break y continue. Así los ciclos anidados utilizan la etiqueta del ciclo que corresponde. Antes de un salto se emiten las salidas de los ámbitos que se abandonan. Si el salto sale de un try, también se retira su manejador. return termina la activación actual y no necesita recorrer los bloques restantes.

En switch se evalúa una sola vez la expresión y se compara con cada case. Después se salta al caso coincidente o al default. Los casos se escriben en orden y permiten continuar al caso siguiente hasta encontrar break. Conservamos la regla booleana de switch de la fase anterior. continue dentro de un switch sigue apuntando al ciclo que lo contiene, si existe.

Asignación y reciclaje de temporales

Cada función utiliza un TemporaryPool. Cuando necesita un temporal, toma uno de la lista de libres o crea uno nuevo. Después de emitir la instrucción que consume un valor, devuelve ese temporal al pool. El temporal que contiene el resultado permanece ocupado hasta que otra instrucción lo utiliza.

No liberamos los argumentos mientras se calculan los siguientes, ni el receptor de un acceso mientras se obtiene su índice o valor. También conservamos los valores que deben sobrevivir a un salto. Los registros de cada función muestran cuántos temporales distintos se crearon y cuántos estuvieron ocupados al mismo tiempo. El algoritmo reduce los temporales necesarios durante el recorrido, aunque no pretende hacer una optimización global ni asignar registros físicos del procesador.

Funciones recursivas y closures

Las funciones de un ámbito se registran antes de traducir las sentencias ordinarias. Esto permite llamadas recursivas y llamadas entre funciones del mismo ámbito. Cada cuerpo queda en una sección separada del TAC. Los argumentos se evalúan de izquierda a derecha y luego se emiten juntos mediante param. La instrucción call consume la cantidad indicada de argumentos y return devuelve el resultado.

```
param 10
param 2
%t1 = call sumar, 2
print %t1
```

Cada llamada crea su propia activación, con parámetros, locales y temporales independientes. En factorial, por ejemplo, la llamada que procesa n conserva sus valores mientras se calcula factorial(n - 1). Al terminar esa llamada se vuelve al llamador y se completa la multiplicación pendiente.

Una closure guarda una etiqueta de función y una referencia al entorno donde fue definida. Las variables capturadas se comparten por referencia, de modo que sus modificaciones siguen disponibles en llamadas posteriores. El entorno permanece vivo después del retorno mientras una closure lo necesite. Dos llamadas a una función que crea closures producen entornos distintos.

```
function crearContador(inicio: integer): Closure {
    function siguiente(): integer {
        inicio = inicio + 1;
        return inicio;
    }
    return siguiente;
}
let contador = crearContador(10);
print(contador());
print(contador());
```

Este ejemplo produce 11 y 12. Closure es una anotación adicional de la fase 2 para permitir que una función retorne otra función. Aprovecha la regla existente de tipos con nombre y propaga la firma de la función conocida para validar las llamadas. No implementamos una gramática general de tipos de funciones ni polimorfismo de funciones de orden superior.

Clases objetos y listas

Cada clase tiene un descriptor con su nombre, clase base, tamaño de instancia, atributos y etiquetas de métodos. newobj crea la instancia y ejecuta los inicializadores desde la clase base hacia la derivada. Después se obtiene e invoca el constructor correspondiente. El acceso a atributos utiliza load_field y store_field.

La instrucción method produce una closure ligada al objeto. Su receptor viaja como @this, separado de los parámetros explícitos. La búsqueda del método comienza en la clase dinámica de la instancia y continúa por sus bases. Esto permite heredar métodos y utilizar una implementación de la clase derivada cuando corresponde.

Las listas se crean con newlist y se consultan o modifican mediante load_index y store_index. Los índices comienzan en cero y se expresan en elementos. El futuro backend podrá convertirlos a desplazamientos en bytes. Las listas multidimensionales se representan como listas que contienen referencias a otras listas. foreach evalúa la colección una vez, obtiene su longitud y recorre los índices.

Tabla de símbolos y registros de activación

Extendimos cada símbolo con dirección, desplazamiento, tamaño, etiqueta y clase de almacenamiento. La dirección incluye el entorno para distinguir variables con el mismo nombre. La etiqueta identifica una función o clase. En los atributos, el desplazamiento corresponde a su posición dentro del objeto; en parámetros y locales corresponde a su posición dentro del modelo de registro.

Utilizamos palabras abstractas de 8 bytes. Los valores simples y las referencias ocupan una palabra. Los objetos y listas se consideran datos de heap, mientras las activaciones organizan los datos de las llamadas. El tamaño de una clase representa el tamaño de sus instancias, incluyendo los atributos heredados y una palabra de cabecera. Los métodos pertenecen al descriptor y no ocupan un atributo adicional en cada objeto.

Parte del registro	Desplazamiento	Uso
Enlace de control	0 bytes	Identificar la activación del llamador.
Enlace de acceso	8 bytes	Acceder al entorno de definición de la función.
Dirección de retorno	16 bytes	Continuar la ejecución después de la llamada.
Valor de retorno	24 bytes	Representar el resultado de la función.
Receptor this	32 bytes, si es método	Conservar la referencia al objeto receptor.
Parámetros y locales	Posiciones siguientes	Reservar espacios distintos para los símbolos.
Temporales	Después de los símbolos	Reservar las posiciones requeridas por el pool.

Los bloques de una misma función utilizan el mismo modelo de registro, con espacios distintos para sus símbolos. Cada inicializador de atributos tiene su propio registro. El programa global también conserva un diseño para el reporte. La tabla extendida puede consultarse en la terminal y en la pestaña de registros del IDE.

Estos desplazamientos son posiciones del modelo del compilador, no direcciones reales de la memoria de Python. El intérprete materializa las llamadas con entornos y una pila de ejecución. El enlace de control corresponde al llamador y el enlace de acceso al entorno léxico capturado. Esa diferencia permite ejecutar una closure desde un lugar distinto al lugar donde fue definida.

Instalación y ejecución

Se necesita Python 3.10 o superior y Tkinter para abrir la interfaz. El compilador no requiere Lark, ANTLR ni Java. Desde la carpeta del repositorio se prepara el entorno y se instalan las dependencias de prueba con estos comandos de PowerShell:

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe scripts\ide.py
```

Al utilizar directamente el ejecutable del entorno no es necesario activarlo. La primera compilación genera el lexer y la tabla SLR en generated. Las siguientes ejecuciones utilizan la caché mientras las especificaciones del lenguaje no cambien.

En el IDE se abre un archivo .cps o se selecciona un ejemplo. Compilar o F5 realiza el análisis y genera el TAC. Las pestañas muestran el árbol, los símbolos, los errores, los cuádruplos y los registros de activación. Ejecutar TAC permite comprobar la salida. Exportar TAC guarda una representación de texto o un JSON con instrucciones y metadatos.

Para generar y ejecutar el ejemplo de expresiones desde la terminal:

```
.\.venv\Scripts\python.exe scripts\compiscript.py `
    examples\fase2_expresiones.cps --tac --ejecutar
```

Para revisar funciones, closures y registros de activación:

```
.\.venv\Scripts\python.exe scripts\compiscript.py `
    examples\fase2_funciones_closures.cps --tac --registros --ejecutar
```

Para revisar objetos, herencia, listas y símbolos extendidos:

```
.\.venv\Scripts\python.exe scripts\compiscript.py `
    examples\fase2_objetos_listas.cps --tac --simbolos --ejecutar
```

Para guardar la salida en texto o en JSON:

```
.\.venv\Scripts\python.exe scripts\compiscript.py `
    examples\fase2_control.cps --salida salida.tac
.\.venv\Scripts\python.exe scripts\compiscript.py `
    examples\fase2_control.cps --salida salida.json
```

Las opciones --tac, --cuadruplos, --registros, --salida y --ejecutar activan la fase 2. Sin estas opciones se conserva el comportamiento de la fase 1. La opción --limite permite establecer cuántas instrucciones puede ejecutar como máximo el intérprete.

Pruebas y resultados

La batería contiene 107 pruebas. Conservamos las pruebas anteriores y agregamos verificaciones del TAC que comparan resultados esperados. Se cubren precedencia, orden de evaluación, cortocircuito, asignaciones, saltos, recursión, closures, métodos, herencia y listas. También se comprueban la CLI, la exportación y la limpieza de salidas anteriores del IDE. Una prueba regenera el lexer y parser sin una caché previa.

```
.\.venv\Scripts\python.exe -m pytest -q
```

Ejemplo	**Resultado esperado**
basico.cps	7
clases_listas.cps	Toby ladra, 11 y 90.
fase2_expresiones.cps	14, 14.0 y correcto.
fase2_control.cps	total correcto y 8.
fase2_funciones_closures.cps	120, 11, 12 y 21.
fase2_objetos_listas.cps	24 y 29.
fase2_error_ejecucion.cps	Mensaje de índice fuera de rango capturado y continuación.
errores.cps	Diagnósticos semánticos sin generar TAC.

Los errores léxicos, sintácticos y semánticos bloquean la generación. Los avisos no la bloquean. Los cuádruplos pueden conservar la ubicación del origen para reportar errores de ejecución. El intérprete comprueba división entre cero, índices negativos o fuera de rango, profundidad de llamadas y límite de instrucciones. try/catch permite capturar errores de ejecución; estos se distinguen de los errores detectados al compilar.

La terminal devuelve código 0 cuando termina correctamente, 1 ante errores de compilación o ejecución y 2 cuando no puede leer o escribir los archivos solicitados. Si una compilación falla, el IDE limpia el TAC anterior para que no se confunda con el resultado del código actual.